

```
//Exception as Throwable
try [
    throw new Exception("Something");
] catch (Throwable $t) [
    echo "Throwable:
    ".$t->getMessage().PHP_EOL;
]
```

3. Accurate Type Declarations

PHP has always been a weak typed language. It has never been mandatory to declare the type of variable; PHP would get it done for you by itself based on the value you assigned. But, at times, this would also result in some unintended and unexpected return values. To overcome this scenario, PHP 7 brings in Type Declarations so that you can specify the type of variable being set instead of depending on PHP to set it automatically for you.

PHP 7 has added two types of declarations for this purpose – Scalar Type Declarations and Return Type Declarations.

Scalar Type Declarations

Scalar Type Declarations comes in two forms – coercive and strict. Scalar types are available in four flavors - **int**, **float**, **string**, and **bool**. Scalar type-declarations, by default, are coercive. In case of coercive, PHP would change the original type of variable to match the one specified by the type-declaration, by itself. This gives you, as a developer, less control over your code and can make the code difficult to understand.

```
function sum(float $a, float $b) [
    return $a + $b;
]
sum(2, "1abc");
```

Consider the above line of PHP code which uses coercive scalar type declaration. What would you expect the **sum** method to return? Go ahead, try this piece of code and you will be amazed with the result returned. Because, it is coercive, PHP would change **int(2)** to **float(2.0)** and **string "1abc"** to **float(1.0)** and return a certain notice. The **sum** method would return **float(3)** as the output.

You can overcome this situation in PHP 7, with strict scalar type declaration. Strict mode is enabled on per-file basis by a single declare directive at the top of the file.

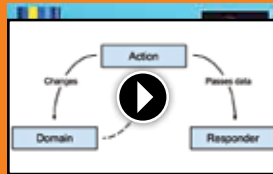
```
declare(strict_types=1);
```

Note that, this directive would also affect the function's return type.

```
declare(strict_types=1);
function sum(float $a, float $b) [
    return $a + $b;
```

*pointers

>>Interesting PHP developer videos



*Introduction - Action Domain Responder

>>A refinement of the MVC which is tuned for the web. Here's how Action-Domain-Responder is better than MVC.

<http://dgit.in/PhpADR>



*Build durable APIs

>>How do you build stable, evolvable and durable APIs for your PHP web application that'll last you for years.

<http://dgit.in/DurableAPIs>



*App level caching

>>Here's how you can implement an efficient and easy to use Application level caching for your PHP web applications.

<http://dgit.in/PhpCache>



*OWASP Top 10

>>OWASP releases it's list of Top 10 security flaws each year. Here are some of the top issues that you should be aware of.

<http://dgit.in/PHPOWASP>



```
]
    sum(2, "1abc");
```

Consider the above line of PHP code which uses strict scalar type declaration. In the above piece of code, for the first parameter PHP will widen the **int** variable by adding a **.0** after the **int** and for the second parameter PHP will produce a **Fatal error: Uncaught TypeError: Argument 2 passed to sum() must be of the type float, string given.**

Return Type Declaration

Like Scalar type, Return Type Declaration also comes in two forms – coercive and strict. Return type declaration lets you specify the type of value which will be returned. In the coercive mode, returned values will be modified to the correct type. In strict mode, if the returned value is not of the correct type, it shall throw a **TypeError**.

```
function sum(float $a, float $b) : float [
    return $a + $b;
]
sum(2,1);
```

The above piece of code is in coercive mode and note that though the parameters passed to the **sum** method are of **int** type, PHP would modify the values to return the result in **float** type.

```
declare(strict_types=1);
function sum(float
$a, float $b) : float [
    return $a + $b;
]
sum(2,1);
```

This piece of code is in strict mode and note that because the parameters passed to the **sum** method are of **int** type and the expected type is **float**, PHP would return you with **Fatal error: Uncaught TypeError: Return value of sum() must be of the type float, integer returned.**

4. Adds Anonymous Class

Other languages in the league of PHP, like C# and Java already had a big support and usage of Anonymous classes. PHP introduced the concept of Anonymous Classes with its latest edition. Anonymous classes are basically classes without names. These are generally used in scenarios where you need to use a class just once during execution or don't need the class to be documented. The object instantiation for an Anonymous class is same as instantiating an object for a names class. However, the best part about Anonymous classes is that you can instantiate an object and define a class in a single line.